

3-Day Task Overview & Usefulness for OnePayslip

Overview of Tasks Completed in 3 Days

This document summarizes the setup, development, and integration tasks completed over the first 3 days of the OnePayslip payroll application project. The goal was to set up the development environment, scaffold both frontend and backend systems, connect them, and configure a continuous integration (CI) pipeline. These foundational steps are crucial for building a scalable, maintainable, and production-ready payroll system.

Day 1: Environment Setup & Frontend Scaffold

- Attended kickoff meeting to understand goals, code standards, and repo structure.
- Installed essential tools: Git, GitHub, Node.js, VS Code, PostgreSQL.
- Cloned and initialized the monorepo template.
- Ran `npm install` in /frontend and /backend.
- Created a basic frontend using Next.js and TypeScript.
- Rendered "Hello OnePayslip" on localhost:3000.
- Committed changes to the 'setup/hello-setup' and 'feature/hello-apps' branches.

Usefulness: Provided a solid foundation for scalable frontend development and ensured consistent tooling across the team.

Day 2: Backend Setup & API Creation

- Set up an Express.js backend server.
- Verified backend running at localhost:4000 with route returning "Hello Payslip".
- Reviewed database schema and created a 'businesses' table with fields (id, name, created_at).
- Implemented GET /businesses API route returning an empty array.
- Added unit tests to ensure the backend logic was working.

3-Day Task Overview & Usefulness for OnePayslip

Usefulness: Established the core backend service, database connectivity, and API route for retrieving business data, which is essential for managing payroll records.

Day 3: Frontend-Backend Integration & CI

- Created frontend route `/businesses.tsx` to fetch data from backend.
- Displayed "No businesses yet" message if backend returned empty data.
- Configured GitHub Actions CI pipeline for linting, building, and testing the app.
- Participated in pair programming to fix UI and backend bugs.
- Committed final changes to 'feature/week1-demo' branch with demo notes.

Usefulness: Demonstrated full-stack data flow and ensured app reliability through CI/CD. This integration validates core use cases like showing business data, a key part of payroll logic.

Overall Usefulness for OnePayslip

- Established a working full-stack foundation for the payroll system.
- Enabled real-time data display through React and Express integration.
- Ensured code reliability with automated CI checks and unit testing.
- Created a maintainable, scalable starting point for future features like employee management, payslip generation, and salary processing.
- Fostered collaboration and knowledge sharing through team sessions and retrospectives.

These tasks ensure OnePayslip is built on solid technical ground and can grow into a production-grade payroll solution with ease.